Available online at www.sciencedirect.com

SciVerse ScienceDirect

Physics Procedia 33 (2012) 315 – 321

Physics Procedia

2012 International Conference on Medical Physics and Biomedical Engineering

Research of Efficient C/S model based File Transfer Method

Xubin Li, Quan Zou, Yi Jiang*

School of Information Science and Technology of Xiamen University, Xiamen city, PR China
**jiangyi@xmu.edu.cn*

Abstract

The one-to-multiple file transfer problem has not been solved under the Linux system. In this paper, we take socket programming to accomplish Client/Server model based file transfer system and develop efficient software which not only support multiple clients and files transferring simultaneously, but also support transmission resuming from breakpoint. Our research focuses on solving the multi-thread concurrency, transmission resuming from breakpoint, thread pool, buffer queue and other aspects of problem. Our work is the basic foundation of FTP servers, file transfer software in local area network and large-scale examination system, etc.

© 2012 Published by Elsevier B.V. Selection and/or peer review under responsibility of ICMPBE International Committee.
Open access under CC BY-NC-ND license.

Keywords: Socket; Multi-Thread; Thread Pool; Transmission resuming from breakpoint; Buffered Queue

1 Introduction

With the rapidly development of the Internet, the number of internet users is growing fast. At the same time, users of Linux operating system extreme more than before. The requirement of file transfer between users is not only eagerly needed, but also the original purpose of network sharing. Therefore, efficient file transfer software in Linux is a problem needed to resolve immediately. Stability and efficiency are most important demands for the file transfer software based on C/S model. But it's hard to prevent transmission error happening while transfer server under high load. For this reason, how to avoid server wasting much time in the retransmission affects us to choose a suitable protocol. As for transport layer of TCP/IP protocol, UDP protocol is faster than TCP protocol in the single file transmission. However, while facing multiple users and files under heavy load and complicated network environment, the pressure from retransmission and timeout rises dramatically[1].

Our research employed TCP protocol as the fundamental of socket programming and built reliable connection-oriented transmission[2]. Meanwhile, the practicability of software was considered at first. 1) In order to achieve goals that one to multiple transfer and multiple files

1875-3892 © 2012 Published by Elsevier B.V. Selection and/or peer review under responsibility of ICMPBE International Committee.
Open access under CC BY-NC-ND license. doi:10.1016/j.phpro.2012.05.068

transferring simultaneously, multi-thread concurrency was introduced. 2) With regard to large file, resuming from breakpoint method was adopted, which made the retransmission without from beginning after connection broke down. 3) With respect to positively relieving the load of server side, our research took advantages of thread pool and buffer queue to optimize data transfer.

2 System design

2.1 Design of communication protocol

Socket was first proposed as the specification of BSD, and now has become into TCP/IP network programming standards under the Linux operating system. It defines the communication specifications between two computers, and shields the differences between the underlying communication and specific operating system, to provide a flexible platform for the communication between server and client[3].

The principle of specific socket programming is shown in Figure 1 as follow:

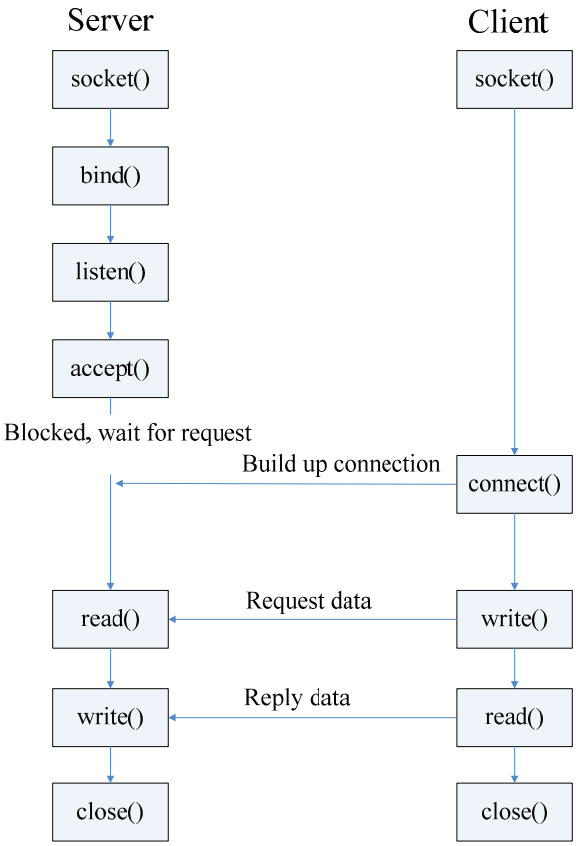

Fig 1 Socket programming

Socket is a sharp weapon for the network programming. It's our best choice to employ socket to realize the transport protocol for our research. There are some main steps of server side on socket communication process as follows: establish socket interface, bind socket interface, monitor connection, accept connection request, communicate between both sides and close socket interface[4]. Client side needn't the step of monitoring the connection and client can connect a remote socket interface directly. What we need care is to ensure a reliable and efficient communication between

both sides. In this study, communication packages include file attribute package sent by client (Head Package), data package sent by both sides (Data Package), and ending signal package sent by client (End Package). There are two kinds of response signal: confirmation signal of Data Package and receive mode signal (agree, refuse, and resume) sent by server. The specific implementation as follows:

(1) Structure *BasePack* has two member variants: *B_Type* which is defined as character type is used to distinguish Data Package types and *B_Length* which is defined as integer type is employed to save the file's total size and actual size of the current data package.

(2) Extended packet is a structure named *ExtendPack* that includes *BasePack* structure variant *E_BP* and character type array *E_DATA* whose size is MAX_DATA_SIZE. *E_DATA* is applied to storage practical data.

2.2 Achievement of multi-thread concurrency and thread pool

To achieve the concurrency for communication, we can adopt multi-process or multi-thread technology. But thread is a powerful tool to improve the code response and performance. Compared to multi-process, multi-thread has two important advantages: more efficient and sharing memory space[1]. Process is not flexible and also consumes much resource, while thread costs a little and system kernel needs not copy the process's memory space or file descriptors, etc. As a result, creating thread saves CPU time and its speed is several times faster than creating process. According to above reason, our research employed multi-thread concurrency.

The specific concurrency is shown in Figure 2 as follow:

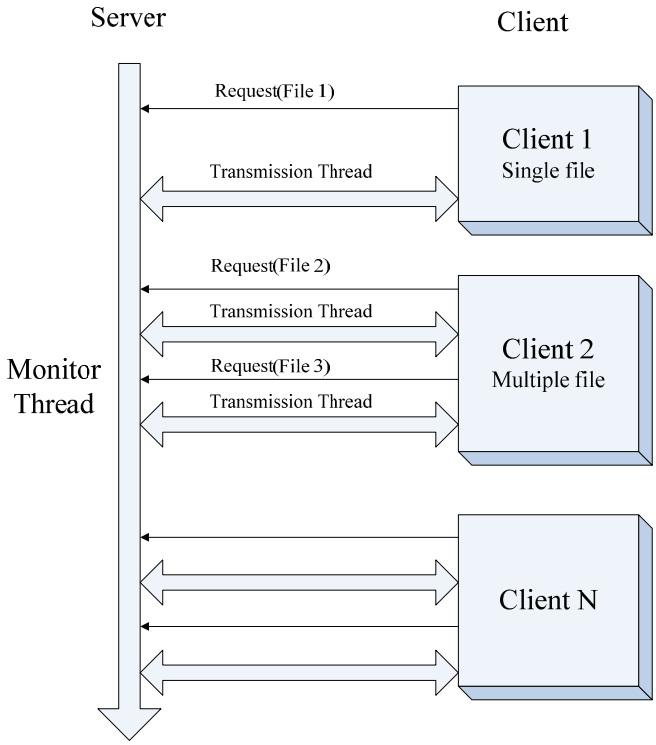

Figure 2 Multi-thread transfer model

To realize the concurrent transmission of server side, we need exploit system resource as much fully as possible, and it's essential to improve the efficiency. The traditional multi-thread technology

creates a new thread by calling *pthread_create* function, while calling *pthread_exit* function to exit thread at the end. However, the actual application shows that creating and destroying thread frequently will consume a large amount of system resource, which encumbers server seriously while under heavy load. Therefore, thread pool can relieve the pressure on server in theory[5].

Thread pool is mainly composed of some following components: *MainThread*, *WorkThread* and *TaskQueue*. *Mainthread* maintains a *TaskQueue* and is responsible for waking a *WorkThread* in the situation of idle blocked up to execute the first task of the queue. The specific achievement will be shown as follows:

(1) Define Structure *ThreadPool_Task* for describing task message. Members of the *ThreadPool_Task* include a pointer pointed to the next task, the address of callback function of *pthread_create*, and parameters of callback function.

(2) Init thread pool *ThreaPool*. This step has three operations including set up listening thread *MainThread,* init *N WorkThread*, and init *TaskQueue*.

(3) Create a *TaskStruct* for each accepted transmission and insert the *TaskStruct* into tail of *TaskQueue*. When the arrival of a new task activates conditional-lock, *MainThread* polling all *WorkThread* and wake up one usable blocked *WokThread* to carry out transmission of the head of *TaskQueue*. After finish the transmission, *WorkThread* block itself and wait for next time been waken up.

2.3 Design of Resuming from Breakpoint Technology (RBT)

When transmission is interrupted (off-line or turn off the computer, etc.), it always means to transmit again from the beginning next time while system doesn't support RBT. But with RBT, transmission can resume from the breakpoint. By saving time, RBT improves the efficiency of transmission[6]. It is much difficult to maintain the connection from beginning to end during file transmission in such complex network environment. Regardless of the incompatible hardware or the quality of network, the existence of discrepancy among nodes in network would lead to errors when data is transmitted. Therefore, it would be time-consuming to retransmit a large file and retransmission will reduce our efficiency if there is no RBT. Thus resuming broken transmission can improve not only the efficiency and reliability of transmission, but also enhance the robustness of network.

Consequently, it is indispensable to achieve the function of RBT for file transfer software.

In this paper, we take the .part breakpoint log file, which is generated when abnormal or positive quit happens, to achieve transmission resuming from the breakpoint. These files have the amount of received data and transmission time in store. RBT mainly involves three components as follows: (1) the server will judge whether the transmission is resuming-type when receiving each request from the client. After getting file name, the server will inquire whether the .part related to data file exists. If it exists, transmission will be judged as an unfinished transmission and a resume request; then the server will send the breakpoint location to the client. The file pointer in the client directly shifts to the breakpoint to transmit data. (2) When the server receives End packet from the client, it will judge the state of transmission. If it is true, which means the process of file transmission is completed, else suspended. (3) During data transfer process, a timeout occurs by without receiving message from the client. If there is no response after signal is sent by the server to the client three times, RBT will be activated. While completing file transmission, .part file will be cleaned automatically.

2.4 Design of Buffer Queue

Time cost is a realistic problem and it's serious in data preparation for transmission. We can make the following assumptions: if the size of Data packet is 2KB, software transmits it needs to apply and free each once in memory. That is, to transmit one file of 10MB requires 5*2*1024 times of memory access. Meanwhile, the thread is blocked and waiting when applying for space. Even if covering the buffer instead of freeing it, efficiency of data preparation is quit low. Hence, a buffer queue is designed to replace the operation: apply and free memory, under control of a separate thread, and the efficiency will be greatly improved.

The data structure of buffer queue is a circular queue actually, as shown in Figure 3.

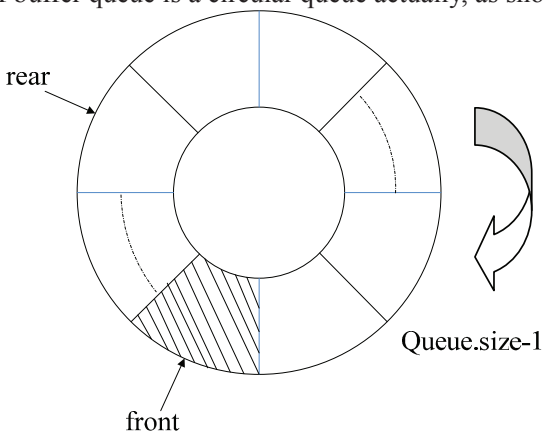

Figure 3 Buffer Queue Model

We employ function *getbuf* to get a buffer: Buf.rear=(Buf.rear+1)%MAX_BUF_SIZE, and function *putbuf* to free a buffer: Buf.front=(Buf.front+1)%MAX_BUF_ SIZE[7].

But one problem occurs: when the queue is full, which means buffer is not enough big, we cannot block the thread but only apply for an extra buffer and free it independently. We can configure a buffer management thread, initialized by MAX_BUF_SIZE as 600KB, namely 300 buffers. And each *WorkThread* has 30 buffers at most, so it basically doesn't need to apply for extra buffer as long as the number of files transmitted simultaneously less than 10.

2.5 User interface design based on GIMP Toolkit (GTK+)

To evaluate the practicability and stability of the software, we developed the file transfer software based on GTK+ UI, which filled the gap of file transfer method in Linux. The shortcut of software is showed in Figure 4.

Figure 4 Screenshot of server side

3. Conclusion

According to the feedback from our software in practical application, software combined above several methods can finish transmission well under high load. However, the C/S based transfer software required technological improvement for the limit of thread on personal computer while attempting to transplant it to server machine.

This paper discussed several practical technologies in file transfer, analyzed the research of efficient C/S model based file transfer method, and designed software that satisfied test requirement. Our research is feasible and software can be downloaded at http://datamining.xmu.edu.cn/main/~xubinli/FTSL.

Acknowledgement

The work was supported by the Natural Science Foundation of China (No.61001013, 61102136, 61001143) and the Natural Science Foundation of Fujian Province of China (No.2011J05158).

3 References

[1] Wang Feng, Luo Jiarong, Study and Application of the Socket Communication Based on Multithreading in Linux[J], *Computer Engineering and Applications*, 2004, 40(16).

[2] W.Richard Stevens, TCP/IP Illustracted Volume 1: The Protocols [M], *Beijing:China Machine Press* , 2000, P293-314.

[3] Wang Yuanyang, Zhou Yuanping, Guo Huanli, The accomplishment of multi-pthread Communication Based on socket model in the Linux[J], *Control and Automation Publication Group*, 2007, 25(5-3), P106-109.

[4] Wu Peixian, Analysis of Socket Programming TCP-based on Linux[J], *Modern Electronics Technique*, 2005, 28(16), P53-55.

[5] Yao Qipeng, Research on Pattern of Thread Pool Based on Linux[J], c*omputer and Modernization*, 2006(12).

[6] Zhu Aimei, Using HTTP to achieve downloading from the point of interruption[J], *World of Entrepreneurs*, 2007, 23(9), P181-183.

[7] W.Richard Stevens, Stephen A.Rago, Advanced Programming in the UNIX Environment (2nd ed) [M], *Post & Telecom Press*, 2006, P200- 300.